

Internship report

Under the supervision of Mr Kerdioui Hichem

Marouane Djabri

September 2026

This report concludes my internship at ENAGEO. It presents a study and performance analysis of Reverse Time Migration (RTM) on GPU architectures. The goal of the study was to measure how much seismic processing can be accelerated by parallel hardware, while verifying the correctness of the results. On the Marmousi2 benchmark, our CUDA implementation migrates a shot 440 to 540 times faster than a single-core CPU reference, and produces the same image to within floating-point rounding. Profiling shows that the computation is limited by memory bandwidth, which explains which optimizations pay off, and a comparison with the Devito framework separates the performance of the kernels from that of the data management.

1. Presentation of the company

1.1. ENAGEO

ENAGEO (Entreprise Nationale de Géophysique) is the Algerian national geophysics company. Established in 1981 and headquartered in Hassi Messaoud, it provides land seismic acquisition services in Algeria and neighbouring countries, that is, it records in the field the seismic data from which the subsurface is later imaged. The quality of its services is recognized for the expertise and field knowledge of its teams.

1.2. Activities

Beyond acquisition, ENAGEO's activities cover the whole geophysical chain:

- *seismic processing*: turning the raw field recordings into images of the subsurface, the step studied in this report;
- *seismic interpretation*: identifying geological structures and potential reservoirs in these images;
- *reservoir characterization*: estimating the physical properties of the rocks inside a reservoir;
- *surface solution services*: topography, geodesy and mapping of the terrain and installations;
- *applied geophysics*: geophysical surveys for engineering and environmental purposes;
- *hydraulic drilling*.

1.3. Mission and vision

ENAGEO's vision is to build fundamental and interdependent platforms, combining strategies, workflows and technologies, able to respond to a large variety of geoscience challenges. Its mission is to be a leading company in the oil and gas industry, offering a wide range of smart solutions together with its partners.

1.4. Products and services

Seismic processing.

- Low Velocity Layer (LVL) modelling: characterization of the weathered layer near the surface, whose slow and irregular velocities distort the seismic signal and must be corrected;
- 2D/3D Kirchhoff pre-stack time migration (KPSTM) processing;

- 2D/3D Kirchhoff pre-stack depth migration (KPSDM) processing;
- velocity model building: estimating the velocity of seismic waves in the subsurface, which every migration method, including RTM, requires;
- seismic data transcription: converting recorded data between storage formats and media.

Reservoir characterization.

- generation of total volumes;
- generation of TOC (Total Organic Carbon) volumes, which indicate the organic content, and therefore the hydrocarbon potential, of the source rocks;
- generation of shale volumes, the proportion of clay-rich rock.

Surface solution services.

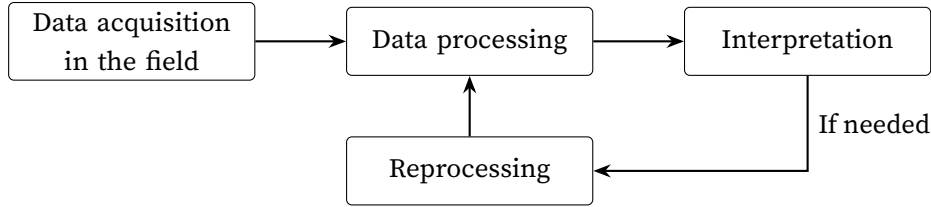
- design and creation of geodetic surveys (precise positioning networks);
- linear and areal topographic surveys;
- topographic surveys by photogrammetry (measurements from photographs);
- well and facility positioning;
- topographic monitoring and control;
- 3D “as built” surveys of facilities, recording their actual geometry after construction;
- subsurface utility detection (buried pipes and cables);
- geographic database design;
- map production.

Geotechnical services. In-situ tests (measurements performed directly in the ground) and laboratory tests.

1.5. Organization and the host department

2. Introduction

Seismic surveying. Seismic surveying is at the heart of oil and gas exploration. It enables geologists, geophysicists and engineers to highlight, interpret and then map features such as structural or stratigraphic traps, where hydrocarbons have accumulated in producible quantities. The workflow starts with data acquisition in the field, followed by processing and interpretation. When needed, the data are reprocessed and interpreted again.



Seismic imaging. Seismic imaging is part of the processing step. It constructs images of the Earth’s subsurface from recorded seismic waves, revealing the geometry of geological layers, faults and other structures. It is computationally intensive: it requires processing large amounts of data and performing complex numerical calculations. This report studies how much one of its most demanding methods, Reverse Time Migration (RTM), can be accelerated on GPUs, while verifying that the results stay correct.

3. Brief description of the seismic imaging process

Principle. Seismic imaging relies on the propagation of elastic waves (Robein 2003). A seismic source (vibrator truck, air gun, dynamite) displaces the particles of the medium, which push on their neighbours, and the disturbance propagates through the subsurface. Where the rock properties change, part of the wave is reflected back to the surface and recorded by receivers. Imaging consists in placing these recorded reflections back at the positions in the subsurface where they occurred.

Migration algorithms. This placement is done by migration algorithms, which differ in how they describe wave propagation and in the resources they require. *Kirchhoff migration* sums the data along computed travel-time curves; it is efficient, but its accuracy is limited where waves follow complex paths. *One-way wave-equation migration* models propagation mainly in one depth direction, which limits its ability to image turning waves and steep structures. *Reverse Time Migration*, the focus of this report, uses the full two-way wave equation: it propagates the source wavefield forward in time and the recorded data backward in time, then combines the two wavefields to form the image. It handles complex propagation paths and steeply dipping structures, at the price of large computing and memory requirements.

4. The computational bottleneck of RTM

To find where the waves were reflected, RTM simulates the propagation of the source wave *forward* in time, propagates the recorded data *backward* in time from the receivers, and looks for the places where the two wavefields meet at the same instant. The imaging step needs the source wavefield at time t , but the receiver wavefield is computed from

the last time step down to the first. The two are only available together if the history of the source wavefield is kept, which turns a computation problem into a *memory* problem: gigabytes of snapshots per shot.

4.1. The RTM algorithm

Let P be the pressure wavefield on a grid of $N = n_x \times n_z$ points (including the absorbing boundary), v the velocity model, Δt the time step, n_t the number of time steps, $w(t)$ the source wavelet, d_s the data recorded for shot s , and k the snapshot interval. The Laplacian ∇_h^2 is a centered finite-difference stencil of order 8, and ω is the absorbing (sponge) taper applied near the edges of the grid.

Algorithm 1 Shot-profile Reverse Time Migration with a zero-lag cross-correlation imaging condition

Require: velocity model v , shots $s = 1, \dots, n_s$ with recorded data d_s

Ensure: migrated image I

```

1:  $I \leftarrow 0$ 
2: for each shot  $s = 1, \dots, n_s$  do
3:   for  $t = 0, \dots, n_t - 1$  do ▷ forward propagation of the source
4:      $P^{\text{next}} \leftarrow \omega(2P - P^{\text{prev}} + (v \Delta t)^2 \nabla_h^2 P)$ , add  $(v \Delta t)^2 w(t)$  at the source
5:     if  $t \bmod k = 0$  then  $S_t \leftarrow P$  ▷ snapshot: the memory cost
6:   end for
7:   for  $t = n_t - 1, \dots, 0$  do ▷ backward propagation of the data
8:      $R^{\text{next}} \leftarrow \omega(2R - R^{\text{prev}} + (v \Delta t)^2 \nabla_h^2 R)$ , add  $(v \Delta t)^2 d_s(t)$  at the receivers
9:     if  $t \bmod k = 0$  then  $I \leftarrow I + S_t \cdot R$  ▷ imaging condition
10:  end for
11: end for
12: return  $I$ 

```

The cost is concentrated in the two update lines (lines 4 and 8): each is executed n_t times per shot and touches all N grid points.

4.2. Computational intensity

One shot performs two full propagations, $W_{\text{shot}} = 2 N n_t$ grid-point updates of about 29 floating-point operations each (order 8). Halving the grid spacing multiplies the number of points by 4 and, through the stability condition, the number of time steps by 2: the work grows as $1/\Delta x^3$. Storing one snapshot every k steps requires $M_{\text{snap}} = (n_t/k) n_x n_z \times 4$ bytes, which grows in the same way. On the Marmousi model with a snapshot every 10 steps, one shot at $\Delta x = 12.5$ m (0.56 million points with the boundary, 2 800 steps) performs 9.0×10^{10} operations, moves 75 GB through memory and stores 0.43 GB of snapshots; at 2.5 m (10.4 million points, 14 000 steps) these become 8.4×10^{12} operations, 7.0 TB and 53 GB.

Arithmetic intensity. Each update reads the current and previous wavefields and the velocity model and writes the new wavefield: 16 to 24 bytes for about 29 operations, i.e. $AI \approx 1.2$ to 1.8 FLOP/byte. An NVIDIA A40 executes about 37 TFLOP/s but delivers only about 0.7 TB/s from its memory: it needs about 54 operations per byte to keep its arithmetic units busy, 30 to 45 times more than RTM provides. RTM is therefore *memory-bound*: its speed is set by memory bandwidth, not by arithmetic capability. This is why GPUs, whose memory bandwidth is several times that of a CPU (578 against 80 GB/s measured on our platform), are the natural target, and why the optimizations studied here aim at moving fewer bytes rather than computing faster.

5. Optimization opportunity

The three nested loops of Algorithm 1 offer parallelism at two levels. *Shots are independent*: each uses its own source and data and only adds its contribution to the image, so shots can run simultaneously on different processors. *Grid points are independent within a time step*: the update of a point at step $t + 1$ reads only values of steps t and $t - 1$, so all N updates of a step, and all N imaging products, can be computed at the same time. *Only time is sequential*. At 12.5 m, each step offers 5.6×10^5 independent updates: a structure that matches a GPU with thousands of cores.

6. Methodology and implementation

6.1. Experimental setup

Platform and tools. The experiments ran on a cloud instance (RunPod) with an NVIDIA A40 GPU (Ampere, 84 SMs, 48 GB, 696 GB/s nominal and 578 GB/s measured bandwidth), 8 cores of an Intel Xeon Gold 6342 and 50 GB of host memory. The engines are written in C++17 with OpenMP and CUDA 12.8; the GPU versions were profiled with NVIDIA Nsight Systems. Nsight Compute could not be used, because the cloud provider blocks access to the GPU performance counters (ERR_NVGPUCTRPERM): the kernel analysis relies on timelines, measured timings and a model of the memory traffic.

Devito. Devito (Louboutin et al. 2019; Luporini et al. 2020) is an open-source finite-difference framework used in research and industry for seismic imaging. The equations are written symbolically in Python, and Devito generates and compiles optimized code for the target machine (OpenACC on GPUs). We implemented the same RTM with Devito and ran it on exactly the same data and parameters. It serves as an independent check of our images and as a state-of-the-art performance reference.

Dataset. The velocity model is Marmousi2 (Martin, Wiley, and Marfurt 2006), a benchmark built on a realistic geological section with layered sediments, faults and a salt-like structure. The shot records are synthetic, computed with our reference engine, so that any difference between an engine’s image and the reference comes from the engine. Table 1 gives the parameters.

TABLE 1. Parameters of the Marmousi2 dataset used for the benchmarks.

Parameter	Value
Model size	17×3.5 km, 1361×281 points, $\Delta x = \Delta z = 12.5$ m
Shots and receivers	12 shots, 341 receivers every 50 m, all at 25 m depth
Recording	$n_t = 2800$ steps of $\Delta t = 1$ ms (2.8 s), Ricker source, $f_0 = 10$ Hz
Numerical scheme	order 8 in space, 2 in time; Cerjan sponge (Cerjan et al. 1985), 50 cells
Snapshots	one every 10 time steps

Correctness criterion. Every image is compared with the image of the CPU reference. An engine is accepted when the relative L_2 difference is below 10^{-5} and the correlation above 0.9999, which corresponds to floating-point rounding.

6.2. Implementation

All engines live in one C++ code base, share the same inputs and outputs, and are selected by name at run time. Every run’s image is compared automatically with the reference: no engine is timed without being checked.

- *CPU reference:* a direct, single-threaded translation of Algorithm 1. It defines the expected image and is reproducible bit for bit (verified on AMD and Intel processors).
- *Optimized CPU engine:* multi-threaded with OpenMP, vectorized, with the absorbing boundary merged into the stencil loop. It produces the same image bit for bit and is the fair CPU baseline.
- *GPU versions v0 to v4:* each version changes one aspect of the previous one, so that the effect of every optimization is measured on its own (Section 6.3).

6.3. The GPU kernels, version by version

Common structure. The wavefields are stored with depth as the fastest index. Each GPU thread updates one grid point, in blocks of 32×8 threads: the 32 threads along z form one warp and read 32 consecutive floats in a single *coalesced* memory transaction. The stencil coefficients live in constant memory. The time loop runs on the CPU, which launches

the kernels of each step; the three wavefields stay in GPU memory and are exchanged by swapping pointers, and only the final image is copied back to the host.

Version 0: the baseline. Version 0 translates the reference directly, with one kernel per operation: the stencil kernel computes the Laplacian from the 8 neighbours in each direction and the new wavefield, a second kernel applies the absorbing boundary to the whole grid, and on imaging steps a third kernel accumulates the image. Since neighbouring values are mostly served by the GPU caches, the stencil moves about 16 bytes per point (read P , P^{prev} , $(v\Delta t)^2$, write P^{next}), and the boundary kernel 20 bytes more: 36 bytes per point and per step.

Version 1: absorbing boundary merged into the stencil. The thread that computes P^{next} already holds it and P in registers; version 1 applies the damping there, before writing ($p_cur[i] = pc_i * w$; $p_next[i] = pn_i * w$;). The separate kernel disappears, the traffic falls from 36 to 24 bytes per point, and the image is unchanged, since the damping coefficient is exactly 1 where sources and receivers are injected.

Version 2: imaging merged into the backward stencil. On imaging steps, the backward stencil kernel accumulates $I \leftarrow I + S_t \cdot R$ with the value it has just computed, instead of a separate kernel reading R again (receiver positions are finished after the data injection). The gain is small: imaging happens once every 10 steps.

Version 3: shared memory. Each block copies its 32×8 points of P , plus a halo of 4 points on each side, into fast on-chip *shared memory*, from which every neighbour is then read.

Version 4: warp shuffles. The vertical neighbours of a point are held by other threads of the same warp; version 4 reads them from their registers with *shuffle* instructions and keeps the tile only for the horizontal neighbours.

TABLE 2. The five GPU versions: kernels launched per time step, estimated memory traffic, and measured GPU time per step (Nsight Systems, NVIDIA A40, 67 200 time steps).

Version	Change	Kernels/step	Bytes/point	Time/step
v0	one thread per point, one kernel per operation	3.05	36	33.3 μ s
v1	absorbing boundary merged into the stencil	2.05	24	27.2 μ s
v2	imaging merged into the backward stencil	2.05	24	27.3 μ s
v3	neighbours read from shared memory	2.05	24	28.4 μ s
v4	vertical neighbours via warp shuffles	2.05	24	28.8 μ s

7. Results and performance analysis

All measurements come from the A40 platform and the dataset of Table 1 unless stated otherwise.

7.1. The migrated images and their correctness

The image. Figure 1 shows the migrated Marmousi2 section after illumination compensation and Laplacian filtering, which remove the low-frequency artefacts of the cross-correlation imaging condition. The structures of the model are visible: layered sediments on the left, faulted blocks dipping through the center between 8 and 11 km, a strong reflector at about 2.3 km depth. The twelve arcs along the top are the acquisition footprint of the twelve shots.

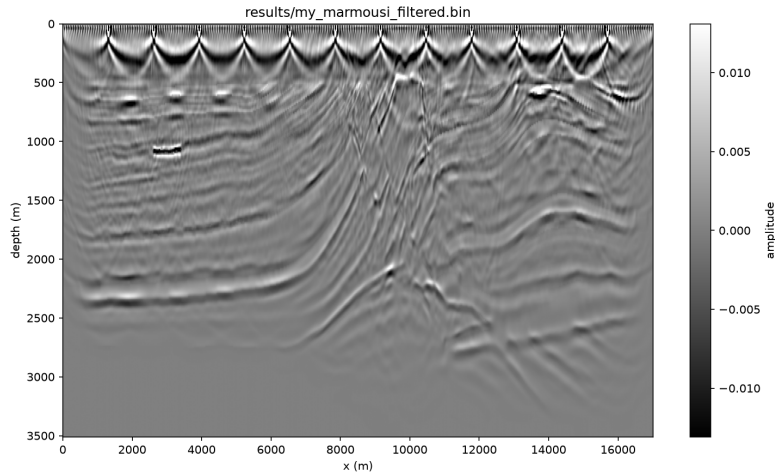


FIGURE 1. Migrated Marmousi2 section (12 shots, $\Delta x = 12.5$ m), after illumination compensation and Laplacian filtering.

Every version produces the same image. Table 3 compares each engine’s image with the reference on two GPUs of different generations. The optimized CPU engine is bit-identical. The GPU images differ by a relative L_2 norm of about 10^{-5} , with a correlation of exactly 1: the signature of floating-point rounding, since the GPU compiler contracts a multiplication and an addition into one fused multiply-add, and small differences accumulate over the 67 200 time steps. A programming error produces differences of 10^{-3} or more. On the Blackwell GPU, v3 and v4 land just above the 10^{-5} threshold; the largest local difference is 0.53×10^{-5} of the image’s peak, near the shallow sources, with no structure resembling a reflector.

TABLE 3. Difference between each engine’s image and the CPU reference image.

Engine	Relative L_2 difference		Correlation
	A40 (Ampere)	RTX PRO 6000 (Blackwell)	
Optimized CPU	0 (bit-identical)	—	1
v0	4.3×10^{-6}	4.3×10^{-6}	1.0000000
v1	4.2×10^{-6}	7.3×10^{-6}	1.0000000
v2	4.6×10^{-6}	7.5×10^{-6}	1.0000000
v3	8.5×10^{-6}	1.45×10^{-5}	1.0000000
v4	6.7×10^{-6}	1.06×10^{-5}	1.0000000

Comparison with Devito. Devito’s image of the same data shows the same reflectors at the same positions: correlation 0.95 after the display filter, 0.69 on the raw images, where the two codes’ different amplitude scaling weighs most. The remaining differences come from independent choices (absorbing boundary, source interpolation); the agreement confirms that our reference solves the intended problem.

7.2. Speed-up over the CPU

Table 4 gives the time needed to migrate the 12 shots, from the first to the last shot, including all data transfers.

A shot that takes 84.7 s on one CPU core, and 41.6 s on 8 cores, takes 0.16 to 0.19 s with our GPU versions: a speed-up of **440 to 540** over the reference and **215 to 266** over the optimized CPU engine. Devito, on the same GPU, is 99 and 49 times faster. The CPU baseline is an 8-core cloud instance, so these speed-ups compare one A40 with this instance. v2 appears faster than v1 only in the total: their propagation times are equal (1.87 s), and v1 spends extra time once before the first shot.

TABLE 4. Time to migrate the 12-shot Marmousi dataset, throughput, and speed-up relative to the two CPU engines (Xeon Gold 6342, 8 cores; NVIDIA A40).

Engine	Total (s)	Shot (s)	Throughput (Gpts/s)	Speed-up vs	
				CPU ref.	opt. CPU
CPU ref. (1 core)	1016.5	84.7	0.037	1	—
Opt. CPU (8 cores)	498.9	41.6	0.075	2.0	1
Devito (GPU)	10.2	0.853	3.7	99	49
Ours, v0	2.32	0.193	16.2	439	215
Ours, v1	2.13	0.178	20.0	476	234
Ours, v2	1.88	0.156	20.0	542	266
Ours, v3	1.95	0.162	19.3	523	256
Ours, v4	1.97	0.164	19.0	516	253

7.3. Where the GPU time goes

Dividing the total time of each kernel recorded by Nsight Systems by the 67 200 time steps gives the cost of one time step (Figure 2).

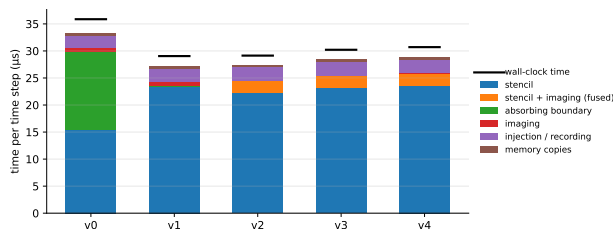


FIGURE 2. GPU time of one time step for each version, broken down by kernel (Nsight Systems, NVIDIA A40). The black marks give the wall-clock time per step.

Merging kernels is the decisive optimization. In v0, the separate absorbing-boundary kernel costs 14.4 μ s per step, almost as much as the stencil (15.4 μ s), although it performs one multiplication per point: both kernels are limited by memory traffic. Merging them (v1) removes one full pass over the grid: 23.5 μ s instead of 29.8, and the time per step drops from 33.3 to 27.2 μ s (−18 %).

Shared memory and shuffles bring nothing. At 20 billion updates per second and 24 bytes per update, v1 already moves about 83 % of the measured bandwidth. v3 and v4 reduce repeated reads of neighbouring values, but these reads were served by the caches: they add instructions and synchronization without reducing the traffic that limits the kernel, and are slightly slower (28.4 and 28.8 μ s).

The CPU barely keeps up. Launching the kernels of a step costs the CPU about 22 μ s, against 27 μ s of GPU work; the GPU is busy 93 to 94 % of the time.

8. Scaling the workload

The Marmousi dataset is small by industrial standards (0.56 million points, 0.16 s per shot on the GPU). To study realistic sizes, the same model was resampled from 12.5 m down to 1.25 m, the way the workload grows in production. The source frequency is raised in proportion ($f_0 = 10 \text{ Hz} \times 12.5/\Delta x$) so that every grid resolves the wave with the same accuracy, the time step is reduced for stability, and every dataset records 2.8 s. Each halving of Δx multiplies the work per shot by 8: the finest grid represents about 700 times the work of the original (Table 5). The snapshot interval is kept at 10 *time steps*: a first campaign with a fixed 10 ms interval aliased the images at fine grids (correlation 0.36 between the 5 ms and 10 ms images at 2.5 m) and was corrected.

8.1. Throughput as the grid grows

TABLE 5. The scaled datasets and the time per shot and throughput of the two codes (NVIDIA A40). Memory: snapshots, one every 10 time steps. OOM: the run exceeded the available memory.

Δx (m)	n_t	Points	Memory	Ours, v1		Devito (GPU)	
				s/shot	Gpts/s	s/shot	Gpts/s
12.5	2 800	0.38 M	0.4 GiB	0.156	20.1	0.853	3.7
6.25	5 600	1.5 M	3.2 GiB	0.994	21.1	3.59	5.8
5	7 000	2.4 M	6.2 GiB	1.83	21.6	6.61	5.9
3.75	9 334	4.2 M	14.7 GiB	4.08	22.1	12.7	7.0
2.5	14 000	9.5 M	49.7 GiB	13.0	22.4	OOM	—
1.25	28 000	38 M	397 GiB	97.9	22.8	OOM	—

Our engine reaches the bandwidth limit. As the work grows by a factor of 700, the time per shot grows by only 630: the throughput *increases* from 20.1 to 22.8 billion updates per second, because the fixed cost of launching kernels becomes negligible. At 24 bytes per update, the kernel then moves about 550 GB/s, 95 % of the measured bandwidth (Figure 3). v3 runs at the same speed as v1 on every grid, even at 38 million points, far beyond the 6 MB cache.

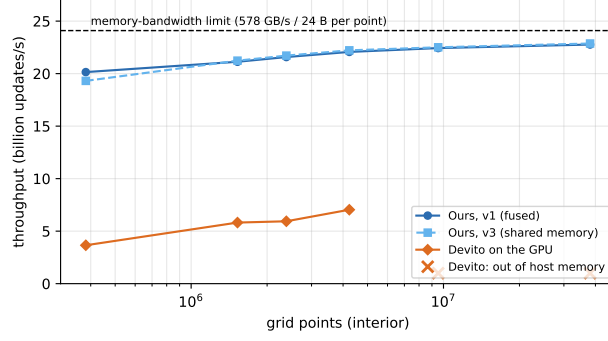


FIGURE 3. Throughput as a function of the number of grid points (NVIDIA A40). The dashed line is the limit set by the measured memory bandwidth.

The stencil order costs nothing. At 2.5 m, raising the order from 4 to 12 leaves our throughput unchanged (22.4, 22.4 and 22.6 Gpts/s), although it triples the arithmetic: the most direct signature of a memory-bound kernel. Devito slows down at high orders (16.0 to 12.0 Gpts/s from order 4 to 16).

8.2. The memory limit

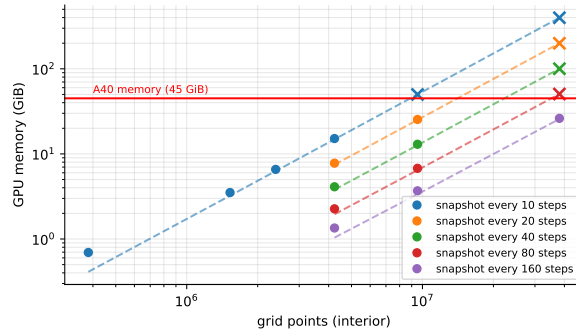


FIGURE 4. GPU memory needed as a function of the grid size, for several snapshot intervals. Dots: measured. Dashed lines: model. Crosses: runs that exceeded the memory of the A40 (red line).

The measured memory follows the model (Figure 4): with a snapshot every 10 steps it grows as $1/\Delta x^3$, from 0.7 GiB at 12.5 m to 15.2 GiB at 3.75 m. At 2.5 m it would need about 50 GiB, more than the 45 GiB available: this is the limit of one A40. Saving fewer snapshots moves the limit, at a price. At 3.75 m, one snapshot every 20 steps leaves the image unchanged (correlation 0.9999986) for half the memory; every 40 steps the correlation falls to 0.86, and beyond, the image is destroyed (0.47 and 0.43), because the snapshots no longer sample the highest frequencies (aliasing). At 1.25 m, only one snapshot every 160 steps fits: problems of this size need several GPUs, or techniques such as checkpointing

or wavefield compression. Devito keeps its snapshots in host memory instead, so its limit is the 50 GB of RAM of the machine.

8.3. Impact on a seismic survey

Projected to 1000 shots at $\Delta x = 2.5$ m (CPU times extrapolated from 12.5 m), a survey would take 2180 hours (91 days) with the CPU reference and 1073 hours (45 days) with the optimized CPU engine, against **3.7 hours** on one A40 with our engine (5.1 with Devito) and less than an hour on 8 GPUs. For a processing team, this is the difference between weeks and a working day.

9. Devito versus our engine: faster kernels, slower pipeline

End to end, our engine is three to five times faster than Devito. Nsight Systems separates the time spent computing from the rest, and this separation reverses the conclusion: *Devito's stencil kernel is faster than ours*, by 7 % at 12.5 m and by 23 to 29 % from 6.25 m onwards (Figure 5a): at 3.75 m, one time step takes 165.6 μ s with Devito against 209.7 μ s with our v1 kernel (28.9 against 22.8 Gpts/s).

Devito's kernel moves fewer bytes per point. Our kernel reproduces the CPU reference exactly, which damps both wavefields at every step: it writes P and P^{next} , 24 bytes per point. Devito damps only the new wavefield and writes only P^{next} : 20 bytes. The ratio $24/20 = 1.2$ is close to the ratio of the kernel times at 3.75 m (1.27), and both kernels run at the bandwidth limit ($22.8 \times 24 \approx 550$ and $28.9 \times 20 \approx 580$ GB/s).

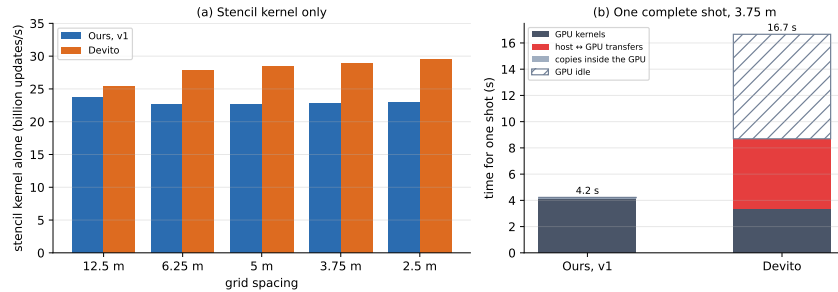


FIGURE 5. (a) Throughput of the stencil kernel alone: Devito is faster on every grid. (b) One complete shot at 3.75 m, as recorded by Nsight Systems: our GPU computes almost all the time, while Devito's GPU spends most of the shot transferring data or idle.

Devito's pipeline moves the saved wavefield over PCIe. Of our 4.2 s shot at 3.75 m, 4.1 s are spent in kernels, and almost nothing crosses the host-GPU link. Devito, in its default configuration, keeps the saved wavefield (17.9 GB per shot at 3.75 m) in host memory:

Nsight Systems records 55 GB of transfers per shot at 10 to 12 GB/s, about 50 times slower than the GPU memory: 5.3 s per shot, more than Devito’s kernels (3.4 s), and the GPU is idle for much of the rest. This is consistent with its lower power draw (115 against 241 W at 3.75 m) and its failure at 2.5 m for lack of host memory.

What this comparison teaches. Damping only the new wavefield would cut our traffic by 20 %; keeping the saved wavefield in GPU memory would remove most of Devito’s transfers. The end-to-end ranking reflects data management choices, not the quality of the generated code.

10. Conclusion

Main findings. *The results are correct:* every GPU version reproduces the reference image to within floating-point rounding (relative difference below 1.5×10^{-5} on two GPU generations), and Devito images the same reflectors. *The GPU changes the scale of the problem:* a shot is migrated 440 to 540 times faster than with the CPU reference and 215 to 266 times faster than with the optimized CPU engine, and a 1000-shot survey at 2.5 m goes from 91 days to 3.7 hours. *RTM is limited by memory bandwidth:* the kernels use up to 95 % of the measured bandwidth and less than 2 % of the arithmetic capability, so merging kernels is the only clear gain (–18 % per step), while shared memory, warp shuffles and higher stencil orders change nothing. *Memory, not speed, limits the problem size:* one A40 reaches its limit between 3.75 and 2.5 m. Finally, Devito’s faster kernel but slower pipeline shows that *kernels and data management are separate problems.*

Limitations and future work. The GPU counters were not accessible, the problem is 2D and acoustic, the data are synthetic, and a single GPU was used. Next steps: 20 bytes per point, CUDA Graphs against the launch overhead, checkpointing or compression and several GPUs against the memory limit, and 3D.

More generally, this work shows that the performance of a seismic imaging code on a GPU depends less on the arithmetic it performs than on the data it moves: the number of bytes per grid point, and where the saved wavefield lives.

References

- Cerjan, Charles, Dan Kosloff, Ronnie Kosloff, and Moshe Reshef. 1985. “A nonreflecting boundary condition for discrete acoustic and elastic wave equations.” *Geophysics* 50 (4): 705–708.
- Louboutin, Mathias, Michael Lange, Fabio Luporini, Navjot Kukreja, Philipp A. Witte, Felix J. Herrmann, Paulius Velesko, and Gerard J. Gorman. 2019. “Devito (v3.1.0): an embedded domain-specific language for finite differences and geophysical exploration.” *Geoscientific Model Development* 12 (3): 1165–1187.

- Luporini, Fabio, Mathias Louboutin, Michael Lange, Navjot Kukreja, Philipp Witte, Jan Hückelheim, Charles Yount, Paul H. J. Kelly, Felix J. Herrmann, and Gerard J. Gorman. 2020. "Architecture and Performance of Devito, a System for Automated Stencil Computation." *ACM Transactions on Mathematical Software* 46 (1): 6:1–6:28.
- Martin, Gary S., Robert Wiley, and Kurt J. Marfurt. 2006. "Marmousi2: An elastic upgrade for Marmousi." *The Leading Edge* 25 (2): 156–166.
- Robein, Etienne. 2003. *Velocities, Time-imaging and Depth-imaging in Reflection Seismics: Principles and Methods*. Houten, The Netherlands: EAGE Publications.